

```

import { Button } from "@components/ui/button";
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@comp
import { Input } from "@components/ui/input";
import { Label } from "@components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@
import { Switch } from "@components/ui/switch";
import { Textarea } from "@components/ui/textarea";
import { AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent, A
import { Tooltip, TooltipContent, TooltipProvider, TooltipTrigger } from "@compo
import {
  Grid, Upload, Download, FileCheck, Palette, Undo, Redo, Save, Clock, Calendar,
  ChevronLeft, ChevronRight, Copy, AlertTriangle, TrendingUp, ListChecks, Chevron
} from "lucide-react";
import { Badge } from "@components/ui/badge";
import { Progress } from "@components/ui/progress";
import { FatigueRiskBadge } from "@components/crew/FatigueRiskBadge";
import { type Crew, type Vessel, MONTHS, chunks, useHoursOfRestData } from "@fea
import { useToast } from "@hooks/use-toast";
import React from "react";

// — Module-level constants (never reconstructed per render) —————
const HOUR_W = 24;
const HDR_H = 26;
const CELL_H = 18;
const GRID_COLS = `110px repeat(24, ${HOUR_W}px) 75px 75px` as const;
const HOURS = Array.from({ length: 24 }, (_, i) => i);

export function HoursOfRestGrid() {
  const { toast } = useToast();

  const {
    meta, setMeta, rows, csv, setCsv, mode, setMode,
    history, historyIndex, saveStatus, viewMode, setViewMode, weekOffset, setWeek
    selectedDay, setSelectedDay, liveCheck, setLiveCheck, showSummary, setShowSum
    customRestStart, setCustomRestStart, customRestEnd, setCustomRestEnd,
    monthsToCopy, setMonthsToCopy, monthsToRemove, setMonthsToRemove, isDragging,
    crew, vessels, filteredCrew, isVesselSelected, isReadyForActions,
    compliance, summaryStats, displayRows,
    undo, redo, startDrag, onDrag, exportCSV, importCSV, clearAll,
    applyCustomRestToAllDays, copyMonthToYear, removeMonths, upload, runCheck, ex
  } = useHoursOfRestData();

  // — Derived / memoised —————
  const selectedCrewMember = React.useMemo(

```

```

    () => crew.find((c) => c.id === meta.crew_id),
    [crew, meta.crew_id],
  );

// — beforeunload guard —————
React.useEffect(() => {
  const handler = (e: BeforeUnloadEvent) => {
    if (saveStatus === "unsaved") {
      e.preventDefault();
      e.returnValue = "";
    }
  };
  window.addEventListener("beforeunload", handler);
  return () => window.removeEventListener("beforeunload", handler);
}, [saveStatus]);

// — Save success toast —————
const prevSaveStatus = React.useRef(saveStatus);
React.useEffect(() => {
  if (prevSaveStatus.current === "saving" && saveStatus === "saved") {
    toast({
      title: "Rest hours saved",
      description: `Saved for ${selectedCrewMember?.name ?? "crew member"} - ${
    });
  }
  prevSaveStatus.current = saveStatus;
}, [saveStatus, selectedCrewMember, meta.month, meta.year, toast]);

return (
  <div className="space-y-6">

    {/* — Save status badge ————— */}
    <div className="flex items-center justify-end gap-2">
      {saveStatus === "saved" && (
        <Badge variant="outline" className="bg-emerald-50 text-emerald-700 border-1">
          <Save className="w-3 h-3 mr-1" />Saved
        </Badge>
      )}
      {saveStatus === "saving" && (
        <Badge variant="outline" className="bg-blue-50 text-blue-700 border-blue-1">
          <Clock className="w-3 h-3 mr-1 animate-spin" />Saving...
        </Badge>
      )}
      {saveStatus === "unsaved" && (
        <Badge variant="outline" className="bg-amber-50 text-amber-700 border-amber-1">
          <AlertTriangle className="w-3 h-3 mr-1" />Unsaved changes
        </Badge>
      )}
    </div>
  </div>
);

```

```

    })
  </div>

  { /* — Setup card ————— */ }
  <Card>
    <CardHeader>
      <CardTitle>Setup</CardTitle>
      <CardDescription>Select vessel and crew member to view or edit their ho
    </CardHeader>
    <CardContent>
      <div className="space-y-6">

        { /* Step 1 - vessel */ }
        <div className="space-y-2">
          <Label htmlFor="vessel-select" className="text-base font-semibold f
            <span className="flex items-center justify-center w-6 h-6 rounded
              Select Vessel
            </Label>
          <Select value={meta.vessel_id || "all"} onChange={(value) => s
            <SelectTrigger data-testid="select-vessel-grid" className="h-11">
              <SelectValue placeholder="Choose a vessel" />
            </SelectTrigger>
            <SelectContent>
              <SelectItem value="all">All Vessels (includes unassigned crew)<
                {vessels.filter((v: Vessel) => v.id).map((vessel: Vessel) => (
                  <SelectItem key={vessel.id} value={vessel.id}>{vessel.name} (
                )))}
              </SelectContent>
            </Select>
            { !isVesselSelected && (
              <p className="text-sm text-muted-foreground">Select a vessel or "
            ) }
          </div>

          { /* Step 2 - crew */ }
          <div className="space-y-2">
            <Label htmlFor="crew-select" className="text-base font-semibold fle
              <span className="flex items-center justify-center w-6 h-6 rounded
                Select Crew Member
              </Label>
            <Select
              value={meta.crew_id}
              onChange={(value) => setMeta({ ...meta, crew_id: value })}
              disabled={!isVesselSelected}
            >
              <SelectTrigger
                data-testid="select-crew-grid"

```

```

        className={`h-11 ${!isVesselSelected ? "opacity-50 cursor-not-a
    >
        <SelectValue placeholder={!isVesselSelected ? "Select vessel fi
    </SelectTrigger>
    <SelectContent>
        {filteredCrew.map((member: Crew) => (
            <SelectItem key={member.id} value={member.id}>{member.name} -
        )))}
    </SelectContent>
    </Select>
    {!isVesselSelected && (
        <p className="text-sm text-muted-foreground">Crew selection will
    )}
    {isVesselSelected && filteredCrew.length === 0 && (
        <p className="text-sm text-amber-600 dark:text-amber-400">No crew
    )}
    </div>

    {/* Step 3 - time period */}
    <div className="space-y-3">
        <Label className="text-base font-semibold flex items-center gap-2">
            <span className="flex items-center justify-center w-6 h-6 rounded
            Select Time Period
        </Label>
        <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
            <div className="space-y-2">
                <Label className="text-sm font-normal">Month</Label>
                <Select value={meta.month} onValueChange={(value) => setMeta({
                    <SelectTrigger data-testid="select-month-grid"><SelectValue /
                    <SelectContent>
                        {MONTHS.map((m) => <SelectItem key={m.label} value={m.label
                    </SelectContent>
                </Select>
            </div>
        <div className="space-y-2">
            <Label className="text-sm font-normal">Year</Label>
            <Input
                type="number"
                placeholder="Year"
                value={meta.year || 2025}
                onChange={(e) => setMeta({ ...meta, year: Number(e.target.val
                data-testid="input-year-grid"
            />
        </div>
    </div>
    </div>
    </div>

```

```

    { /* Ready banner */ }
    { isReadyForActions && (
      <div className="p-3 bg-emerald-50 dark:bg-emerald-950 border border
        <div className="flex items-center justify-between">
          <p className="text-sm text-emerald-700 dark:text-emerald-300 fl
            <CheckCircle2 className="h-4 w-4" />
            <span>
              Ready to edit hours of rest for{" "}
              <strong>{selectedCrewMember?.name}</strong> ({meta.month} {
            </span>
          </p>
          <div className="flex items-center gap-2">
            <span className="text-xs text-muted-foreground">Fatigue Risk:
              <FatigueRiskBadge
                crewId={meta.crew_id}
                crewName={selectedCrewMember?.name}
                showScore
              />
            </div>
          </div>
        </div>
      ) }
    </div>
  </CardContent>
</Card>

{ /* — Summary card ————— */ }
{ showSummary && isReadyForActions && (
  <Card className="border-2 border-blue-200 dark:border-blue-800">
    <CardHeader>
      <div className="flex items-center justify-between">
        <div className="flex items-center gap-2">
          <TrendingUp className="w-5 h-5 text-blue-600" />
          <CardTitle>Compliance Summary</CardTitle>
        </div>
        <Button variant="ghost" size="sm" onClick={() => setShowSummary(fal
          <ChevronUp className="w-4 h-4" />
        </Button>
      </div>
      <CardDescription>Month overview and compliance statistics</CardDescri
    </CardHeader>
    <CardContent>
      <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
        <div className="p-4 bg-gradient-to-br from-emerald-50 to-emerald-10
          <p className="text-sm text-muted-foreground">Compliance Rate</p>
          <p className="text-2xl font-bold text-emerald-700 dark:text-emera
            <Progress value={Number.parseFloat(summaryStats.complianceRate)}

```

```

        <p className="text-xs text-muted-foreground mt-1">{summaryStats.c
    </div>
    <div className="p-4 bg-gradient-to-br from-blue-50 to-blue-100 dark
        <p className="text-sm text-muted-foreground">Avg Rest/Day</p>
        <p className="text-2xl font-bold text-blue-700 dark:text-blue-300
        <p className="text-xs text-muted-foreground mt-1">Total: {summary
    </div>
    <div className="p-4 bg-gradient-to-br from-amber-50 to-amber-100 da
        <p className="text-sm text-muted-foreground">Violations</p>
        <p className="text-2xl font-bold text-amber-700 dark:text-amber-3
        <p className="text-xs text-muted-foreground mt-1">{summaryStats.c
    </div>
    <div className="p-4 bg-gradient-to-br from-purple-50 to-purple-100
        <p className="text-sm text-muted-foreground">Longest Work</p>
        <p className="text-2xl font-bold text-purple-700 dark:text-purple
        <p className="text-xs text-muted-foreground mt-1">Continuous peri
    </div>
    </div>
    </CardContent>
</Card>
)}

{!showSummary && isReadyForActions && (
    <Button variant="outline" size="sm" onClick={() => setShowSummary(true)}
        <ChevronDown className="w-4 h-4 mr-2" />Show Summary Dashboard
    </Button>
)}

/* — View & Edit Controls ————— */
<Card>
    <CardHeader>
        <div className="flex items-center justify-between flex-wrap gap-4">
            <CardTitle>View & Edit Controls</CardTitle>
            <div className="flex items-center gap-2">
                {/* View mode toggle */}
                <div className="flex items-center gap-1 border rounded-md p-1">
                    <Button
                        variant={viewMode === "month" ? "default" : "ghost"}
                        size="sm"
                        onClick={() => setViewMode("month")}
                        data-testid="button-view-month"
                    >
                        <Calendar className="w-4 h-4 mr-1" />Month
                    </Button>
                    <Button
                        variant={viewMode === "week" ? "default" : "ghost"}
                        size="sm"

```

```

        onClick={() => { setViewMode("week"); setWeekOffset(0); }}
        data-testid="button-view-week"
      >
        <ListChecks className="w-4 h-4 mr-1" />Week
      </Button>
      <Button
        variant={viewMode === "mobile" ? "default" : "ghost"}
        size="sm"
        onClick={() => setViewMode("mobile")}
        data-testid="button-view-mobile"
      >
        <Smartphone className="w-4 h-4 mr-1" />Mobile
      </Button>
    </div>

    { /* Undo / Redo */ }
    <div className="flex items-center gap-1">
      <TooltipProvider>
        <Tooltip>
          <TooltipTrigger asChild>
            <Button
              variant="outline"
              size="sm"
              onClick={undo}
              disabled={historyIndex <= 0}
              title="Undo (Ctrl+Z)"
              data-testid="button-undo"
            >
              <Undo className="w-4 h-4" />
            </Button>
          </TooltipTrigger>
          <TooltipContent side="bottom">
            {historyIndex <= 0
              ? "Nothing to undo"
              : `Undo (${historyIndex} step${historyIndex !== 1 ? "s" : ""} ago)`}
          </TooltipContent>
        </Tooltip>
      </TooltipProvider>

      <TooltipProvider>
        <Tooltip>
          <TooltipTrigger asChild>
            <Button
              variant="outline"
              size="sm"
              onClick={redo}
              disabled={historyIndex >= history.length - 1}
            >
              <Redo className="w-4 h-4" />
            </Button>
          </TooltipTrigger>
          <TooltipContent side="bottom">
            {historyIndex >= history.length - 1
              ? "Nothing to redo"
              : `Redo (${history.length - historyIndex} step${history.length - historyIndex !== 1 ? "s" : ""} ago)`}
          </TooltipContent>
        </Tooltip>
      </TooltipProvider>
    </div>
  )}

```

```

        title="Redo (Ctrl+Y) "
        data-testid="button-redo"
      >
        <Redo className="w-4 h-4" />
      </Button>
    </TooltipTrigger>
    <TooltipContent side="bottom">
      {historyIndex >= history.length - 1
        ? "Nothing to redo"
        : `Redo (${history.length - 1 - historyIndex} step${histo
      </TooltipContent>
    </Tooltip>
  </TooltipProvider>
</div>
</div>
</div>
</CardHeader>
<CardContent>
  {/* Week navigation */}
  {viewModel === "week" && (
    <div className="flex items-center justify-between mb-4 p-3 bg-slate-5
      <Button
        variant="outline"
        size="sm"
        onClick={() => setWeekOffset(Math.max(0, weekOffset - 1))}
        disabled={weekOffset === 0}
      >
        <ChevronLeft className="w-4 h-4" />Previous Week
      </Button>
      <span className="font-medium">Week {weekOffset + 1} of {Math.ceil(r
      <Button
        variant="outline"
        size="sm"
        onClick={() => setWeekOffset(Math.min(Math.ceil(rows.length / 7)
        disabled={weekOffset >= Math.ceil(rows.length / 7) - 1}
      >
        Next Week<ChevronRight className="w-4 h-4" />
      </Button>
    </div>
  )}

  <div className="flex items-center gap-2">
    <Switch id="live-check" checked={liveCheck} onCheckedChange={setLiveC
    <Label htmlFor="live-check" className="text-sm">Live compliance check
  </div>
</CardContent>
</Card>

```



```

{ /* — Custom Rest Schedule — */ }
<Card>
  <CardHeader>
    <CardTitle className="flex items-center gap-2">
      <Clock className="w-5 h-5" />Custom Rest Schedule
    </CardTitle>
    <CardDescription>Define rest periods and apply to days, months, or enti
  </CardHeader>
  <CardContent>
    <div className="space-y-4">
      { /* Time range */ }
      <div className="space-y-3">
        <Label className="text-sm font-medium">Rest Period Time Range</Labe
        <div className="flex items-center gap-3 flex-wrap">
          <div className="flex items-center gap-2">
            <Label className="text-xs text-muted-foreground whitespace-nowr
            <Input
              type="time"
              value={customRestStart}
              onChange={ (e) => setCustomRestStart(e.target.value) }
              className="w-32"
              data-testid="input-rest-start-time"
            />
          </div>
          <div className="flex items-center gap-2">
            <Label className="text-xs text-muted-foreground whitespace-nowr
            <Input
              type="time"
              value={customRestEnd}
              onChange={ (e) => setCustomRestEnd(e.target.value) }
              className="w-32"
              data-testid="input-rest-end-time"
            />
          </div>
        </div>
        <p className="text-xs text-muted-foreground">Example: 20:00 to 06:0
      </div>

      { /* Apply to current month */ }
      <div className="pt-3 border-t">
        <Label className="text-sm font-medium mb-2 block">Apply to Current
        <div className="flex gap-2 flex-wrap">
          <Button
            size="sm"
            variant="outline"
            onClick={applyCustomRestToAllDays}

```

```

        className="bg-blue-50 hover:bg-blue-100 dark:bg-blue-950 dark:h
        data-testid="button-apply-rest-all-days"
      >
      <Copy className="w-3 h-3 mr-1" />Copy to All Days of {meta.mont
    </Button>
  </div>
</div>

{/* Copy month to year */}
<div className="pt-3 border-t">
  <Label className="text-sm font-medium mb-2 block">Copy Month to Ent
  <div className="space-y-3">
    <div className="p-3 bg-amber-50 dark:bg-amber-950 rounded-lg bord
      <p className="text-xs text-amber-800 dark:text-amber-200 mb-2">
        This will copy the current month's schedule ({meta.month} {me
      </p>
      <div className="grid grid-cols-2 sm:grid-cols-3 md:grid-cols-4
        {MONTHS.map((month) => (
          <label key={month.label} className="flex items-center gap-2
            <input
              type="checkbox"
              checked={monthsToCopy.includes(month.label)}
              onChange={(e) => {
                if (e.target.checked) {
                  setMonthsToCopy([...monthsToCopy, month.label]);
                } else {
                  setMonthsToCopy(monthsToCopy.filter((m) => m !== mo
                }
              }}
              className="rounded"
            </>
            <span>{month.label.slice(0, 3)}</span>
          </label>
        ))}
      </div>
    </div>
    <div className="flex gap-2">
      <Button
        size="sm"
        variant="default"
        onClick={copyMonthToYear}
        disabled={monthsToCopy.length === 0}
        className="bg-green-600 hover:bg-green-700 text-white"
        data-testid="button-copy-month-to-year"
      >
      <Copy className="w-3 h-3 mr-1" />Copy to {monthsToCopy.length
    </Button>
  </div>
</div>

```

```

        <Button size="sm" variant="outline" onClick={() => setMonthsToC
          Select All
        </Button>
        <Button size="sm" variant="outline" onClick={() => setMonthsToC
          Clear
        </Button>
      </div>
    </div>
  </div>

  { /* Remove months */ }
  <div className="pt-3 border-t">
    <Label className="text-sm font-medium mb-2 block">Clear Month Data<
    <div className="space-y-3">
      <div className="p-3 bg-red-50 dark:bg-red-950 rounded-lg border b
        <p className="text-xs text-red-800 dark:text-red-200 mb-2">
          Select months to clear their rest schedule data
        </p>
        <div className="grid grid-cols-2 sm:grid-cols-3 md:grid-cols-4
          {MONTHS.map((month) => (
            <label key={month.label} className="flex items-center gap-2
              <input
                type="checkbox"
                checked={monthsToRemove.includes(month.label)}
                onChange={(e) => {
                  if (e.target.checked) {
                    setMonthsToRemove([...monthsToRemove, month.label])
                  } else {
                    setMonthsToRemove(monthsToRemove.filter((m) => m !==
                  }
                }}
                className="rounded"
              </>
              <span>{month.label.slice(0, 3)}</span>
            </label>
          )}}
        </div>
      </div>
      <Button
        size="sm"
        variant="destructive"
        onClick={removeMonths}
        disabled={monthsToRemove.length === 0}
        data-testid="button-remove-months"
      >
        <AlertTriangle className="w-3 h-3 mr-1" />Clear {monthsToRemove
      </Button>

```

```

        </div>
    </div>
</div>
</CardContent>
</Card>

{/* — Editing Tools ————— */}
<Card className="border-slate-200 dark:border-slate-700 shadow-md">
    <CardHeader className="bg-gradient-to-r from-slate-50 to-slate-100 dark:f
        <CardTitle className="text-lg font-semibold text-slate-800 dark:text-sl
        <CardDescription className="text-slate-600 dark:text-slate-400">
            Click cells to toggle, or use paint mode to drag and fill multiple ce
        </CardDescription>
    </CardHeader>
    <CardContent className="p-6">
        <div className="space-y-6">
            {/* Paint mode info */}
            <div className="p-4 bg-gradient-to-r from-slate-50 to-blue-50 dark:fr
                <div className="flex items-start gap-3">
                    <Palette className="w-5 h-5 text-slate-600 dark:text-slate-400 fl
                    <div className="flex-1">
                        <Label className="font-semibold text-slate-700 dark:text-slate-
                        <p className="text-xs text-muted-foreground">
                            Click to toggle individual cells, or click and drag to toggle
                            <span className="inline-flex items-center gap-1">
                                <span className="w-2.5 h-2.5 bg-emerald-400 rounded-full" /
                                </span>{" "}or{" "}
                                <span className="inline-flex items-center gap-1">
                                    <span className="w-2.5 h-2.5 bg-rose-400 rounded-full" />WO
                                </span>
                            </p>
                        </div>
                    </div>
                </div>

            <div className="space-y-4">
                {/* Save & Verify */}
                <div className="space-y-2">
                    <Label className="text-sm font-medium text-slate-700 dark:text-sl
                    <div className="flex gap-2 flex-wrap">
                        <Button
                            onClick={upload}
                            size="default"
                            disabled={!isReadyForActions}
                            className={`shadow-md transition-all duration-200 ${
                                !isReadyForActions
                                    ? "opacity-50 cursor-not-allowed bg-gray-400 hover:bg-gra

```

```

        : "bg-blue-600 hover:bg-blue-700 text-white hover:shadow-
    }`}
    data-testid="button-upload-grid"
    title={!isReadyForActions ? "Select vessel and crew member fi
>
    <Upload className="w-4 h-4 mr-2" />Save to Database
</Button>
<Button
    onClick={runCheck}
    variant="outline"
    size="default"
    disabled={!isReadyForActions}
    className={`transition-all duration-200 ${
        !isReadyForActions
        ? "opacity-50 cursor-not-allowed border-gray-300 text-gra
        : "border-amber-300 text-amber-700 hover:bg-amber-50 hove
    }`}
    data-testid="button-check-grid"
    title={!isReadyForActions ? "Select vessel and crew member fi
>
    <FileCheck className="w-4 h-4 mr-2" />Check Compliance
</Button>
</div>
</div>

{/* Data Management */}
<div className="space-y-2">
    <Label className="text-sm font-medium text-slate-700 dark:text-sl
    <div className="flex gap-2 flex-wrap">
        <Button
            onClick={loadFromProposedPlan}
            variant="outline"
            size="sm"
            disabled={!isReadyForActions}
            className={`transition-all duration-200 ${
                !isReadyForActions
                ? "opacity-50 cursor-not-allowed border-gray-300 text-gra
                : "border-indigo-300 text-indigo-700 hover:bg-indigo-50 h
            }`}
            data-testid="button-load-proposed-plan"
            title={!isReadyForActions ? "Select vessel and crew member fi
>
            <FileCheck className="w-4 h-4 mr-2" />Load from Schedule
        </Button>
        <Button
            onClick={exportPdf}
            variant="outline"

```

```

        size="sm"
        className="border-purple-300 text-purple-700 hover:bg-purple-
data-testid="button-export-pdf-grid"
        title="Generate PDF report"
    >
        <Download className="w-4 h-4 mr-2" />Export PDF
    </Button>
    <Button
        onClick={exportCSV}
        variant="outline"
        size="sm"
        className="border-cyan-300 text-cyan-700 hover:bg-cyan-50 hov
data-testid="button-export-csv"
        title="Export to CSV file"
    >
        Export CSV
    </Button>
    <Button
        onClick={importCSV}
        variant="outline"
        size="sm"
        className="border-teal-300 text-teal-700 hover:bg-teal-50 hov
data-testid="button-import-csv"
        title="Import from CSV file"
    >
        Import CSV
    </Button>

    { /* Clear All – with confirmation dialog */ }
    <AlertDialog>
        <AlertDialogTrigger asChild>
            <Button
                variant="outline"
                size="sm"
                className="border-slate-300 text-slate-700 hover:bg-slate
data-testid="button-clear-all"
                title="Clear all hours in the grid"
            >
                Clear All
            </Button>
        </AlertDialogTrigger>
        <AlertDialogContent>
            <AlertDialogHeader>
                <AlertDialogTitle>Clear all rest hours?</AlertDialogTitle>
                <AlertDialogDescription>
                    This will clear all rest hours for{" "}
                    <strong>{selectedCrewMember?.name ?? "this crew member"}

```

```

        <strong>{meta.month} {meta.year}</strong>.
        <br />
        You can immediately undo this with the Undo button.
    </AlertDialogDescription>
</AlertDialogHeader>
<AlertDialogFooter>
    <AlertDialogCancel>Cancel</AlertDialogCancel>
    <AlertDialogAction
        onClick={clearAll}
        className="bg-destructive hover:bg-destructive/90"
    >
        Clear All Hours
    </AlertDialogAction>
</AlertDialogFooter>
</AlertDialogContent>
</AlertDialog>
</div>
</div>
</div>
</div>

{/* STCW rules info */}
<div className="mt-6 p-4 bg-slate-100 dark:bg-slate-800 rounded-lg bord
    <div className="flex items-start gap-3">
        <div className="w-6 h-6 bg-blue-100 dark:bg-blue-900 rounded-full f
            <FileCheck className="w-3 h-3 text-blue-600 dark:text-blue-400" /
        </div>
        <div>
            <h4 className="font-medium text-slate-800 dark:text-slate-200 tex
            <ul className="text-xs text-slate-600 dark:text-slate-400 space-y
                <li>• <span className="font-medium">Minimum 10 hours</span> res
                <li>• <span className="font-medium">Minimum 77 hours</span> res
                <li>• <span className="font-medium">Maximum 2 rest blocks</span>
                <li>• <span className="text-indigo-600 dark:text-indigo-400">Ni
            </ul>
        </div>
    </div>
</div>
</div>
</CardContent>
</Card>

{/* — Mobile day-by-day view ————— */}
{viewModel === "mobile" ? (
    <Card className="border-2 border-slate-200 dark:border-slate-700 shadow-l
        <CardHeader className="bg-gradient-to-r from-slate-50 to-slate-100 dark
            <CardTitle className="text-xl font-semibold text-slate-800 dark:text-
            <CardDescription className="text-slate-600 dark:text-slate-400">

```

```

        Optimized for mobile devices - tap time blocks to edit
    </CardDescription>
</CardHeader>
<CardContent className="p-4 space-y-4">
    {displayRows.filter(Boolean).map((r, ri) => {
        const c = compliance[ri];
        const restChunks = chunks(r);
        return (
            <div key={r.date} className="p-4 bg-white dark:bg-slate-900 round
            <div className="flex items-center justify-between mb-3">
                <div>
                    <h4 className="font-semibold text-lg">
                        {new Date(r.date + "T00:00:00Z").toLocaleDateString("en-U
                    </h4>
                    <p className="text-xs text-muted-foreground">{r.date}</p>
                </div>
                <Badge variant={c.dayOK ? "default" : "destructive"} classNam
                    {c.dayOK ? "✓ Compliant" : "✗ Violation"}
                </Badge>
            </div>
            <div className="relative h-12 bg-slate-100 dark:bg-slate-800 ro
                {restChunks.map(([start, end], idx) => {
                    const isRest = (r as Record<string, number | string>)[`h${s
                    const width = ((end - start) / 24) * 100;
                    const left = (start / 24) * 100;
                    return (
                        <div
                            key={`chunk-${start}-${end}`}
                            className={`absolute h-full ${isRest ? "bg-emerald-400"
                            style={{ left: `${left}%`, width: `${width}%` }}
                            title={`${isRest ? "REST" : "WORK"} ${start}:00-${end}:
                        />
                    );
                })}
            <div className="absolute inset-0 grid grid-cols-24">
                {HOURS.map((h) => (
                    <button
                        key={h}
                        onMouseDown={() => startDrag(ri, h)}
                        onMouseEnter={() => isDragging && onDrag(ri, h)}
                        className="border-1 border-slate-300 dark:border-slate-
                    />
                ))}
            </div>
        </div>
        <div className="grid grid-cols-3 gap-2 text-xs">
            <div className="text-center p-2 bg-slate-50 dark:bg-slate-800

```



```

        <p className="text-muted-foreground">Rest Total</p>
        <p className={`font-semibold ${c.restTotal >= 10 ? "text-em
</div>
<div className="text-center p-2 bg-slate-50 dark:bg-slate-800
    <p className="text-muted-foreground">Min 24h</p>
    <p className={`font-semibold ${c.minRest24 >= 10 ? "text-em
</div>
<div className="text-center p-2 bg-slate-50 dark:bg-slate-800
    <p className="text-muted-foreground">Blocks</p>
    <p className={`font-semibold ${c.splitOK ? "text-emerald-60
        {restChunks.filter([a]) => (r as Record<string, number |
    </p>
</div>
</div>
</div>
    );
  }}}
</CardContent>
</Card>
) : (

/* — Main grid (month / week) ————— */
<Card className="border-2 border-slate-200 dark:border-slate-700 shadow-l
  <CardHeader className="bg-gradient-to-r from-slate-50 to-slate-100 dark
    <CardTitle className="text-xl font-semibold text-slate-800 dark:text-
    <CardDescription className="text-slate-600 dark:text-slate-400">
      Click to toggle cells, drag to paint.{ " "}
    <span className="inline-flex items-center gap-1">
      <span className="w-3 h-3 bg-emerald-200 dark:bg-emerald-800 round
    </span>{ " "}•{ " "}
    <span className="inline-flex items-center gap-1">
      <span className="w-3 h-3 bg-rose-200 dark:bg-rose-800 rounded bor
    </span>
    </CardDescription>
  </CardHeader>
  <CardContent className="p-6">

    {/* Legend */}
    <div className="flex flex-wrap items-center gap-x-5 gap-y-2 mb-3 px-1
      <span className="flex items-center gap-1.5 font-medium text-foregro
      <span className="flex items-center gap-1.5">
        <span className="inline-block w-4 h-4 rounded bg-emerald-100 dark
      </span>
      <span className="flex items-center gap-1.5">
        <span className="inline-block w-4 h-4 rounded bg-rose-100 dark:bg
      </span>
      <span className="flex items-center gap-1.5">

```

```

        <span className="inline-block w-4 h-4 rounded border-2 border-ind
    </span>
    <span className="ml-auto italic">Click or drag to toggle</span>
</div>

{/* Grid */}
<div
    className="overflow-x-auto bg-white dark:bg-slate-900 rounded-lg bo
    data-testid="rest-hours-grid"
>
    {/* Sticky header */}
    <div className="sticky top-0 z-10" style={{ display: "grid", gridTe
        <div className="bg-slate-100 dark:bg-slate-800 border-r border-sl
            Date
        </div>
        {HOURS.map((h) => (
            <div
                key={h}
                className="bg-gradient-to-b from-slate-100 to-slate-200 dark:
                style={{ height: HDR_H + 6, lineHeight: `${HDR_H + 6}px`, fon
            >
                {String(h).padStart(2, "0")}
            </div>
        ))}
        <div className="bg-slate-100 dark:bg-slate-800 border-r border-sl
            Rest/24h
        </div>
        <div className="bg-slate-100 dark:bg-slate-800 text-center font-m
            Min24h
        </div>
    </div>

    {/* Data rows */}
    {displayRows.filter(Boolean).map((r, ri) => {
        const c = compliance[viewMode === "week" ? weekOffset * 7 + ri :
        const dayOK = c?.dayOK;
        const actualIndex = viewMode === "week" ? weekOffset * 7 + ri : r

        // Build violation tooltip
        const violationDetail = !dayOK ? (() => {
            const reasons: string[] = [];
            if (c.restTotal < 10) reasons.push(`Rest ${c.restTotal}h < 10h
            if (c.minRest24 < 10) reasons.push(`24h window: ${c.minRest24.t
            if (!c.splitOK) reasons.push("Block split violated");
            return "⚠ " + (reasons.join(" · ") || "Violation");
        })() : "Compliant";
    })

```

```

return (
  <div
    key={r.date}
    role="button"
    tabIndex={0}
    className={`group hover:bg-slate-50 dark:hover:bg-slate-850 t
    onClick={() => setSelectedDay(actualIndex)}
    onKeyDown={(e) => {
      if (e.key === "Enter" || e.key === " ") {
        e.preventDefault();
        setSelectedDay(actualIndex);
      }
    }}
  >
    <div style={{ display: "grid", gridTemplateColumns: GRID_COLS
      { /* Date cell */ }
      <div
        className={`bg-slate-50 dark:bg-slate-800 border-r border
        title={!dayOK && liveCheck ? violationDetail : undefined}
      >
        <span className="text-xs">{r.date.slice(8, 10)}</span>
      </div>

      { /* Hour cells */ }
      { HOURS.map((h) => {
        const v = (r as Record<string, number | string>)[`h${h}`]
        const isRest = v === 1;
        const isNightHour = h >= 20 || h < 6;
        return (
          <div
            key={h}
            role="checkbox"
            aria-checked={isRest}
            aria-label={` ${isRest ? "Rest" : "Work"} at ${String(
              tabIndex={0}
            onMouseDown={(e) => { e.preventDefault(); startDrag(a
            onMouseEnter={() => onDrag(actualIndex, h)}
            onKeyDown={(e) => {
              if (e.key === " " || e.key === "Enter") {
                e.preventDefault();
                startDrag(actualIndex, h);
              }
            }}
            className={`border-r border-b border-slate-200 dark:b
              isRest
                ? "bg-emerald-100 dark:bg-emerald-900 hover:bg-em
                : "bg-rose-100 dark:bg-rose-900 hover:bg-rose-200

```

```

    } ${isNightHour ? "ring-1 ring-inset ring-indigo-300
style={{ width: HOUR_W, height: CELL_H + 2, position:
data-testid={`grid-cell-${actualIndex}-${h}`}
>
    {isRest && (
      <div className="absolute inset-0 flex items-center
        <div className="w-1 h-1 bg-emerald-600 dark:bg-em
        </div>
      </div>
    )}
  </div>
);
}}

{/* Rest/24h total */}
<div
  className={`border-r border-b border-slate-200 dark:borde
    c.restTotal >= 10
      ? "bg-emerald-50 dark:bg-emerald-950 text-emerald-700
      : "bg-rose-50 dark:bg-rose-950 text-rose-700 dark:tex
    }`}
  style={{ fontSize: 11 }}
>
  {c.restTotal}
</div>

{/* Min 24h */}
<div
  className={`border-b border-slate-200 dark:border-slate-7
    c.minRest24 >= 10
      ? "bg-emerald-50 dark:bg-emerald-950 text-emerald-700
      : "bg-rose-50 dark:bg-rose-950 text-rose-700 dark:tex
    }`}
  style={{ fontSize: 11 }}
>
  {c.minRest24.toFixed(0)}
</div>
</div>

{/* Compliance bar */}
<div
  className={`h-1 transition-all duration-300 shadow-sm ${
    dayOK
      ? "bg-gradient-to-r from-emerald-400 to-emerald-600"
      : "bg-gradient-to-r from-rose-400 to-rose-600"
    }`}
  style={{ marginBottom: 2 }}
  title={violationDetail}

```

```

        />
      </div>
    );
  }}}
</div>

{ /* Selected day detail panel */
{selectedDay != null && compliance[selectedDay] && displayRows[selec
  <div className="mt-3 p-3 rounded-lg border bg-blue-50 dark:bg-blue-
    <span className="font-semibold text-blue-800 dark:text-blue-200">
      {displayRows[selectedDay].date}
    </span>
    <span className={compliance[selectedDay].restTotal >= 10 ? "text-
      Rest: <strong>{compliance[selectedDay].restTotal}h</strong>
      {compliance[selectedDay].restTotal < 10 && " ⚠ (min 10h)"}
    </span>
    <span className={compliance[selectedDay].minRest24 >= 10 ? "text-
      24h window: <strong>{compliance[selectedDay].minRest24.toFixed(
    </span>
    <span className={compliance[selectedDay].splitOK ? "text-emerald-
      Block split: <strong>{compliance[selectedDay].splitOK ? "OK" :
    </span>
    <button
      onClick={() => setSelectedDay(null)}
      className="ml-auto text-xs text-muted-foreground hover:text-for
      aria-label="Dismiss day detail"
    >
      x
    </button>
  </div>
  )}
</CardContent>
</Card>
)}

{ /* — CSV mode ————— */
{mode === "CSV" && (
  <Card>
    <CardHeader>
      <CardTitle>CSV Data</CardTitle>
      <CardDescription>Edit raw CSV data (date,h0..h23)</CardDescription>
    </CardHeader>
    <CardContent>
      <Textarea
        className="h-40 font-mono text-sm"
        value={csv}
        onChange={(e) => setCsv(e.target.value)}

```

```
        data-testid="textarea-csv"
      />
      <div className="flex gap-2 mt-2">
        <Button onClick={importCSV} size="sm" data-testid="button-import-cs
          Import & Apply
        </Button>
        <Button onClick={() => setMode("GRID")} variant="outline" size="sm"
          Back to Grid
        </Button>
      </div>
    </CardContent>
  </Card>
)}
</div>
);
}
```